

Assessment

Mid-Level Full Stack Developer

Project Brief:

Real-Time Event Dashboard & API

- **Time Commitment:** 3–4 hours (or a 48-hour take-home window)
- **Deliverables:** Public GitHub Repository (with commit history) and a brief hREADME.md (architecture notes + setup instructions).

Task Scenario

Build a responsive single-page web application that ingests, displays, and filters live user activity logs.

Technical Requirements

1. Backend & API

- Implement a REST or WebSocket API (Node.js/TypeScript, Python/FastAPI, or Go).
- **Endpoints:**
 - POST /api/events: Receive an event payload { id, user_id, event_type, payload, timestamp }. Validate input and save to a local database (SQLite, PostgreSQL, or MongoDB).
 - GET /api/events: Retrieve events with pagination (page, limit) and filter options (event_type, date_range).
 - GET /api/events/analytics: Return aggregate metrics (e.g., event counts per type over the last 24 hours).
- Implement rate limiting (e.g., max 30 requests/minute per IP).

2. Frontend

- Build a UI using React/Next.js or Vue.js with modern CSS (Tailwind CSS or component libraries).
- **Key Features:**

- **Activity Feed:** Display incoming events in real-time or via polling (every 5 seconds).
- **Filters:** Filter feed by event type and search payload text without full page reloads.
- **Analytics Card:** A chart or metric card summarizing top event types.
- **Responsive Design:** Mobile and desktop compatible.

3. Engineering Standards

- Unit or integration tests covering at least the main API endpoints and core utility functions.
- Clean environment variable management (.env.example).
- Structured error handling for invalid payloads and DB connections.
- Containerization: Provide production-ready Dockerfile(s) for both backend and frontend, plus a root docker-compose.yml that orchestrates the services (and database, if using PostgreSQL/MongoDB) with a single command.